

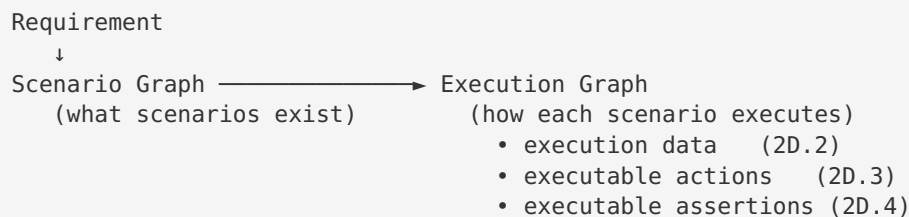
The Execution Graph Contract (frozen)

Status: FROZEN as of Sprint 2D. Architecture review complete — the section model is final. Remaining work is populating the graph, not reshaping it. Changes to the section model below require a design review and a schema-version bump — not an ad-hoc field added to a node.

Scope: This document freezes the canonical shape of `ScenarioNode` / `ScenarioGraph` (`src/graph/scenario-graph.ts`) before Sprint 2D adds executable Actions (2D.3) and executable Assertions (2D.4). It exists so that six months from now nobody drops `resolvedDataset` inside `semantics` , or an assertion inside `metadata` .

1. Why this is now called the Execution Graph

The type is still named `ScenarioGraph` in code (renaming is deferred to avoid churn), but the **mental model has changed**. Through Sprint 2C it described what scenarios exist. After Sprint 2D it will describe exactly how each scenario executes:



Once complete, it is the **core domain model of LevelUp AI** — the single canonical execution model consumed by:

Script Generation ☐ Healing ☐ Replay ☐ Self-Healing ☐ Impact Analysis ☐ Coverage Gap Analysis

Every one of those reads the same node. None re-derives it.

2. The canonical `ScenarioNode` — seven sections

A node is organised into **seven ownership sections**. Every field lives in exactly one section, chosen by the placement rules in §3. This is the frozen contract:

```

ScenarioNode {

    // — 1. IDENTITY ————— immutable, defines "which scenario"
    id                // stable KB scenarioId – the join key for every consumer
    title
    objective

    // — 2. KNOWLEDGE (semantics) ————— immutable, "what the scenario
    MEANS"
    semantics {
        variableUnderTest
        preconditions
        variation
        expectedBehavior

requiredDataRole      // DEPRECATED – a data ROLE ("a registered user"), never a
resolved              // dataset. Migrates to resources.dataRoles in Graph
Schema 2.0.           // Kept here only until `resources` lands so migration
                        // stays additive.
    }

    // — 3. RESOURCES (reserved) ————— immutable, "what the scenario
    NEEDS"
    // REQUIREMENTS ONLY. Resources holds abstract capability requirements, never a
    // resolved/selected value. Right: registered_user, chromium, staging, invent-
    ory_api.
    // Wrong: standard_user, "Chrome 139", "staging-03" – those are resolved values and
    // belong in `execution`. Execution owns the resolved values, forever.
    resources {
        dataRoles      // e.g. ["registered_user"] (the role requirement, NOT
"standard_user")
        services        // e.g. ["inventory_api"] (required backing
services / mocks)
        environment     // required env CLASS (e.g. "staging"), NOT a host like
"staging-03"
        browser         // required browser CAPABILITY (e.g. "chromium"), NOT
"Chrome 139"
        locale          // required locale (e.g. "en-US")
        // featureFlags / device / tenant / oauthClient – natural future members
    }

    // — 4. EXECUTION ————— runtime, "what THIS run actually
    USED"
    // RUNTIME RESOLVED VALUES ONLY. Execution answers exactly one question:
    // "what did this run actually use?" – nothing more.
    // Allowed: resolvedDataset, resolvedBrowser, resolvedLocale, resolvedEnviron-
    ment.
    // NOT allowed: executionOrder, retryPolicy, priority, coverage – those are not
    // "what this run used"; they belong in metadata or elsewhere.
    execution {
        resolvedDataset // 2D.2 – concrete record resolved from re-
sources.dataRoles
        // resolvedBrowser / resolvedLocale / resolvedEnvironment – natural future members
    }

    // — 5. ACTIONS (landed in 2D.3) ————— immutable, "the executable steps"
    actions[] {
        id                // stable identity for this step within the node
        order             // 0-based execution order (array is authoritative)
        action            // navigate | fill | click | check | uncheck | select |

```

```

upload
                                // (STATE-CHANGING verbs ONLY – no `verify`;
assertions are $6)

target                          // CANONICAL element identity (app-neutral semantic key,
e.g.                            // `username` – NOT `email_input` and NOT a raw
locator). The                   // Builder copies this VERBATIM; the Execution
Resolver in                     // Script Gen grounds it to a locator at emit time.
    value?                     // literal, or @dataset.* reference resolved from execu-
tion                           //
    optional?                  // step may be skipped when its target is absent
    }

// — 6. ASSERTIONS (landed in 2D.4) ————— immutable, "the executable expected
outcomes"
    assertions[] {
        id                      // STABLE SEMANTIC identity – `

```

```

coverageType, priority, severity, riskArea, tags,
automationReady, automationComplexity, selectorAvailability,
source, sourceEvidence, grounded,
expectedResults          // human-readable outcomes (superseded for execution by
$6)
dependencies             // typed edges live on the graph; per-node refs here if
needed
metadata                 // confidence, timings, telemetry – never behaviour-bearing
}

```

Note on section slots: `actions[]` (2D.3) and `assertions[]` (2D.4) are now **populated in code**; `resources` remains **reserved, not yet in code**. The node's invariant ("every field must serve ≥ 2 consumers") means each lands in the PR that also adds its first consumer — `actions[]` in 2D.3, `assertions[]` in 2D.4, and `resources` when a second consumer beyond the Dataset Resolver needs it (env/browser/locale requirements). Today the data ROLE requirement still lives in `semantics.requiredDataRole`; it migrates to `resources.dataRoles` when the `resources` section lands.

This document froze where each goes and what it contains so those PRs are pure fills, not re-designs.

3. Placement rules — what belongs in each section

Section	Immutable?	Answers...	Example	Do NOT put here
identity	✓ immutable	Which scenario is this?	<code>id: "auth-neg-wrong-password"</code>	anything run-varying
semantics	✓ immutable	What does it fundamentally mean?	<code>variableUnderTest: "password"</code>	resolved values, locators, requirements
resources	✓ immutable	What does it NEED to run?	<code>dataRoles: ["registered_user"]</code>	resolved/selected values (those are execution)
execution	✗ runtime	What did THIS run actually use?	<code>resolvedDataSet: {username, ...}</code>	anything that changes identity
actions	✓ immutable	What steps execute, in order?	<code>{action: "fill", target: "username"}</code>	resolved values inline (use <code>@dataset.*</code>)
assertions	✓ immutable	What outcomes are verified?	<code>{type: "url", expected: "@page.inventory"}</code>	prose like "login succeeds"; concrete URLs/copy (<code>/inventory</code> , "Bad password")
metadata	✗ diagnostic	How confident / how measured?	<code>confidence: 0.82</code>	anything behaviour-bearing

The three-question separation — the reason `resources` and `execution` are distinct sections:

Question	Section	Example
What does this scenario mean?	<code>semantics</code>	variable under test = password
What does this scenario need?	<code>resources</code>	a registered user, chromium, en-US
What did this run actually use?	<code>execution</code>	standard_user, Chrome 139, en-US

`resources` is the immutable requirement (“I need a registered user”); `execution` is the mutable resolution (“this run used `standard_user`”). They are different abstractions and must never be merged. This is exactly why `requiredDataRole` (the need) and `resolvedDataset` (the resolution) cannot share a section.

The decision test (apply to every new field)

1. **Does it change if the same scenario runs with a different dataset / env / browser?**
 → YES → is it the requirement (immutable need) or the resolved value (this run)?
 → requirement → `resources` ; resolved value → `execution` . NO → continue.
2. **Is it a value the run produces or measures (never an input)?**
 → YES → `metadata` . NO → continue.
3. **Is it a step the browser performs?** → `actions` .
4. **Is it an outcome the test verifies?** → `assertions` .
5. **Does it define what the scenario means independent of any run?** → `semantics` .
6. **Does it identify which scenario this is?** → `identity` .

If a field seems to fit two sections, it is probably two fields. Split it. (The canonical example:

`requiredDataRole` / `dataRoles` is a `resources` requirement; the record it resolves to is `execution` .)

Hard invariants (still enforced)

- **A ROLE requirement is never a resolved dataset.** The data-role requirement (`semantics.requiredDataRole` today, `resources.dataRoles` once `resources` lands) is an immutable NEED. The Dataset Resolver maps role
 → record; the resolved record lands in `execution.resolvedDataset` . A resolved record must **never** appear in `semantics` or `resources` — requirement and resolution are different abstractions.
- **target is a CANONICAL, application-neutral semantic identity — never app vocabulary and never a raw locator.** The KB authors canonical targets (`username`), the Builder copies them VERBATIM (it does NOT translate `username` → `email_input`), and the **Execution Resolver** in Script Gen grounds canonical →
 app selector → Playwright locator at emit time. This keeps the graph framework- and app-neutral: if the app renames a field, only the resolver changes — the graph never regenerates.
- **Actions are STATE-CHANGING verbs ONLY** (`navigate` / `fill` / `click` / `check` / `unchecked` / `select` / `upload`).
 There is deliberately no `verify` action — a scenario’s expected outcomes are Assertions (§6), a separate concern. Actions do; assertions check. Mixing them was explicitly rejected.
- **value may be a @dataset.* reference.** Actions reference execution data symbolically so the same action list is reusable across datasets — the value is bound from `execution.resolvedDataset` at emit time.
- **Assertions use a FROZEN grammar of 11 types** (`url` / `visible` / `hidden` / `enabled` / `disabled` / `checked` / `unchecked` / `text` / `value` / `count` / `attribute`). This is a checkable-property

vocabulary, NOT a scenario vocabulary: there is deliberately no `success`, `failure`, `login`, or `logout`

type — those are scenarios whose outcome decomposes into these primitive checks. Adding a type is a

reviewed schema change, never an ad-hoc edit.

- **Assertion expected may be a SYMBOLIC reference the resolver grounds.** `@page.<name>` resolves to a concrete URL/route and `@messages.<name>` to concrete UI copy — both via the Execution Resolver's App Knowledge map in Script Gen, never in the graph. This keeps the graph application-neutral: the same assertion set works for any app, and rewording a message or renaming a route re-runs only the resolver. For `attribute`, `expected` is encoded `name=value` (e.g. `type=password`). An unresolved reference DEGRADES safely (a `text` check falls back to a visibility check) rather than emitting invented copy.
- **Assertion id is STABLE and SEMANTIC, never positional.** The builder derives `id` from the check's meaning — `<scenarioId>.<type>.<subject>` — not its array index. So a durable consumer (Coverage, Healing, Replay, Analytics) can reference “the login-error text check” by id and keep hitting the same check even after assertions are reordered, inserted, or removed. `order` carries sequence ONLY; identity lives in `id`. This is why `order` is free to change without breaking references, and why the builder must never fall back to `:a:<index>` ids.
- **Assertions store CANONICAL business meaning, never Playwright.** The graph holds `{type:"visible", target:"login_error"}`, never `expect(...)`, `toBeVisible()`, or a CSS locator. Script Gen is a pure `switch(type)` renderer + resolver — it NEVER infers an assertion from prose. The KB authors the check; the graph carries it; the renderer grounds and emits it.
- **Every shared-node field serves ≥2 consumers.** Single-consumer data belongs in that consumer, not on the node. The graph is a contract, not a junk drawer.
- **Identity / semantics / resources / actions / assertions are immutable per scenario.** Only `execution` and `metadata` vary run-to-run. The fingerprint hashes identity + semantics + resources + actions + assertions — never execution or metadata.

4. Schema-version governance

`SCENARIO_GRAPH_SCHEMA_VERSION` (currently `'1.2.0'` — `1.0.0` → `1.1.0` when 2D.3 populated `actions`,

1.1.0 → 1.2.0 when 2D.4 populated `assertions`) is the contract version. Bump it when the shape changes:

- **PATCH** — additive optional field within an existing section, backward-compatible.
- **MINOR** — new section slot populated for the first time (`resources` , 2D.3 `actions` , 2D.4 `assertions`).
- **MAJOR** — a field moves sections, is removed, or changes meaning (requires migration + review).

Persisted graphs record the version they were built with; readers must tolerate older optional-field-absent

graphs (as they already do for `semantics` and `execution`).

5. Migration state (Sprint 2D)

Section	Present in code today	Populated by	First consumer
identity	✓	builder	all
semantics	✓ (optional)	builder ← KB (<code>getScenarioSemantics</code>)	Test Case Lab, Script Gen, Healing, Dataset Resolver
resources	▣ reserved (role req. in <code>semantics.requiredData</code> Role today)	builder ← KB	Dataset Resolver (+ future env/browser consumers)
execution	✓ (optional; <code>resolvedDataset</code>)	builder ← Dataset Resolver	Test Case Lab, Script Gen (2D.2)
actions	✓ (optional)	builder ← KB (<code>getScenarioActionTemplate</code>)	Script Gen (<code>generateFromTestCase</code> + <code>generateFromTestCases</code>)
assertions	✓ (optional)	builder ← KB (<code>getScenarioAssertionTemplate</code>)	Script Gen (<code>emitGraphAssertionLines</code>)
metadata	✓ (scattered)	builder / validator	RTM, telemetry

The rule for the rest of Sprint 2D: additions land in the reserved slots above, in the PR that also adds their consumer. No field is added anywhere else on the node without updating this document and bumping the schema version.

6. Definition of “frozen”

The contract is frozen means:

1. The seven sections and their ownership are fixed. New data goes into an existing section per §3, or is not a node concern.
2. `resources`, `actions[]` and `assertions[]` have a **known, documented shape** before they are implemented — so those PRs populate a pre-agreed slot rather than debating structure mid-sprint.
3. Any deviation is a reviewed schema change, not a silent field.

This is what lets every subsequent sprint be additive: the destination for each new capability is already

decided. **No more structural discussions — only fill the reserved sections.**

The one question every PR from here must answer:

What capability moved into the Execution Graph?

— not — “what new architecture should we invent?” The model is complete enough that the remaining work is

populating the graph, not reshaping it. Do not redesign the model again unless a real product problem

forces it (which is then a reviewed MAJOR schema change, per §4).

Next step: Sprint 2D.5 — DELETE the legacy assertion inference now that the graph owns assertions, making Script Gen a pure renderer with no fallback. (2D.4 — graph owns executable `assertions[]` (`id` / `order` / `type` / `target` / `expected` / `optional`), rendered by Script Gen’s `emitGraphAssertionLines` —

is implemented and in review; 2D.3 — graph owns executable `actions[]` — is in review as PR #274; 2D.2 —

consume `execution.resolvedDataset` — merged as PR #273.)